

Advanced Attention Mechanisms - II

Large Language Models: Introduction and Recent Advances

ELL881 · AIL821



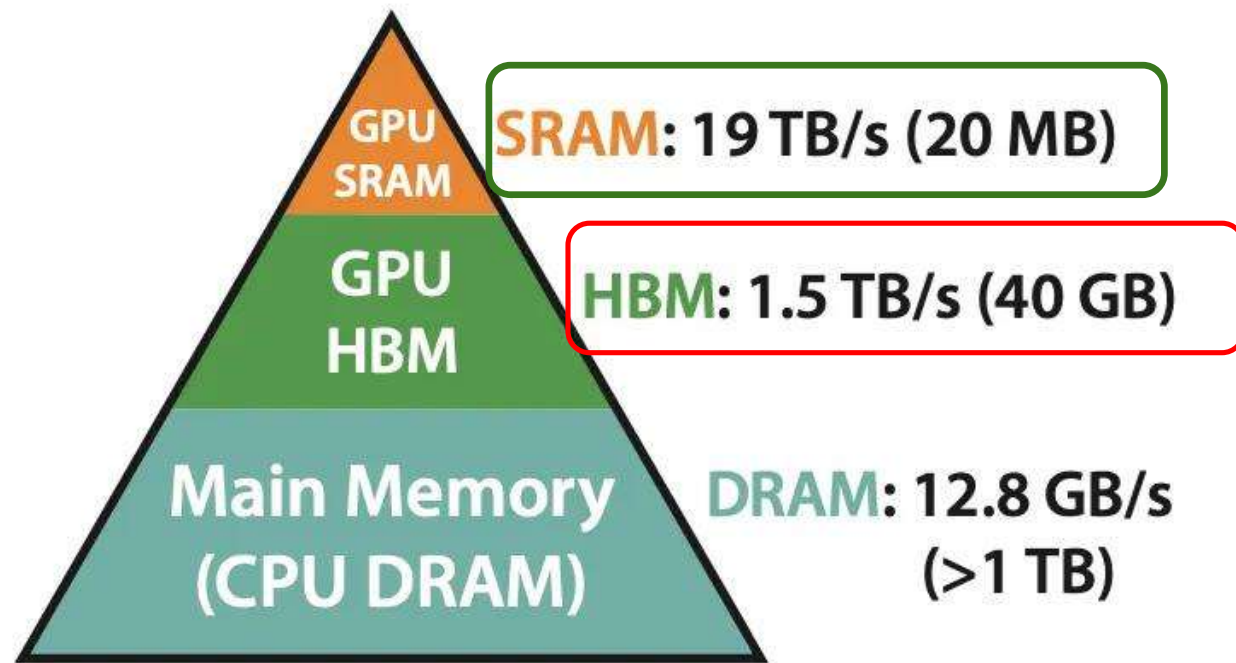
Sourish Dasgupta

Assistant Professor, DA-IICT, Gandhinagar

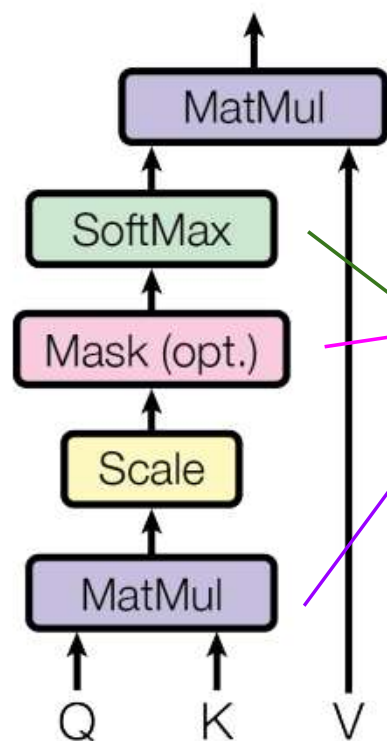
<https://daiict.ac.in/faculty/sourish-dasgupta>

Can we optimize without
performance degradation?

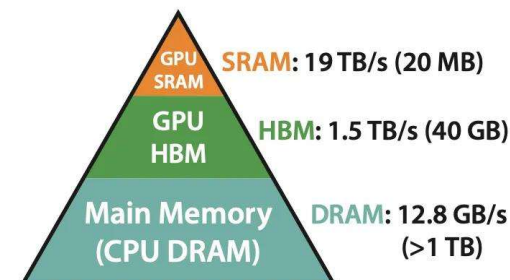
A bit more about the GPU



What was happening so far:

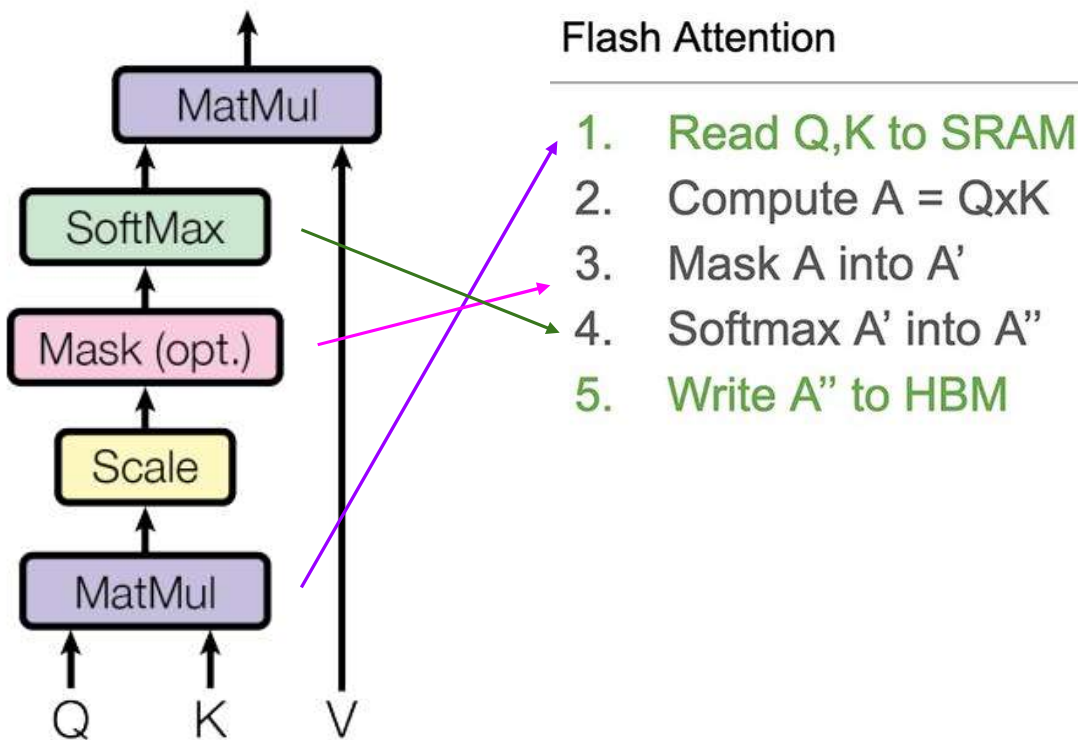


1. Matmul_op (Q,K)
 - a. Read Q,K to SRAM (read-op)
 - b. Compute matmul $A=Q \times K$ (compute-op)
 - c. Write A to HBM (write-op)
2. Mask_op
 - a. Read A to SRAM (read-op)
 - b. Mask A into A' (compute-op)
 - c. Write A' to HBM (write-op)
3. Softmax_op
 - a. Read A' to SRAM (read-op)
 - b. Softmax A' into A'' (compute-op)
 - c. Write A'' to HBM (write-op)





The magic: Fused Kernel (GPU Operations)!



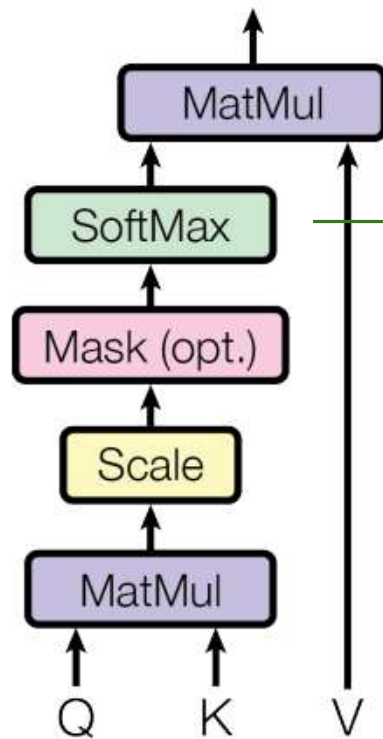
1. Matmul_op (Q, K)
 - a. Read Q, K to SRAM (read-op)
 - b. Compute matmul $A = Q \times K$ (compute-op)
 - c. Write A to HBM (write-op)
2. Mask_op
 - a. Read A to SRAM (read-op)
 - b. Mask A into A' (compute-op)
 - c. Write A' to HBM (write-op)
3. Softmax_op
 - a. Read A' to SRAM (read-op)
 - b. Softmax A' into A'' (compute-op)
 - c. Write A'' to HBM (write-op)



Year: 2022; NeurIPS



The magic does not end here! More optimization



$$s(x_i) = \frac{e^{x_i}}{\sum_{j=1}^N e^{x_j}}$$

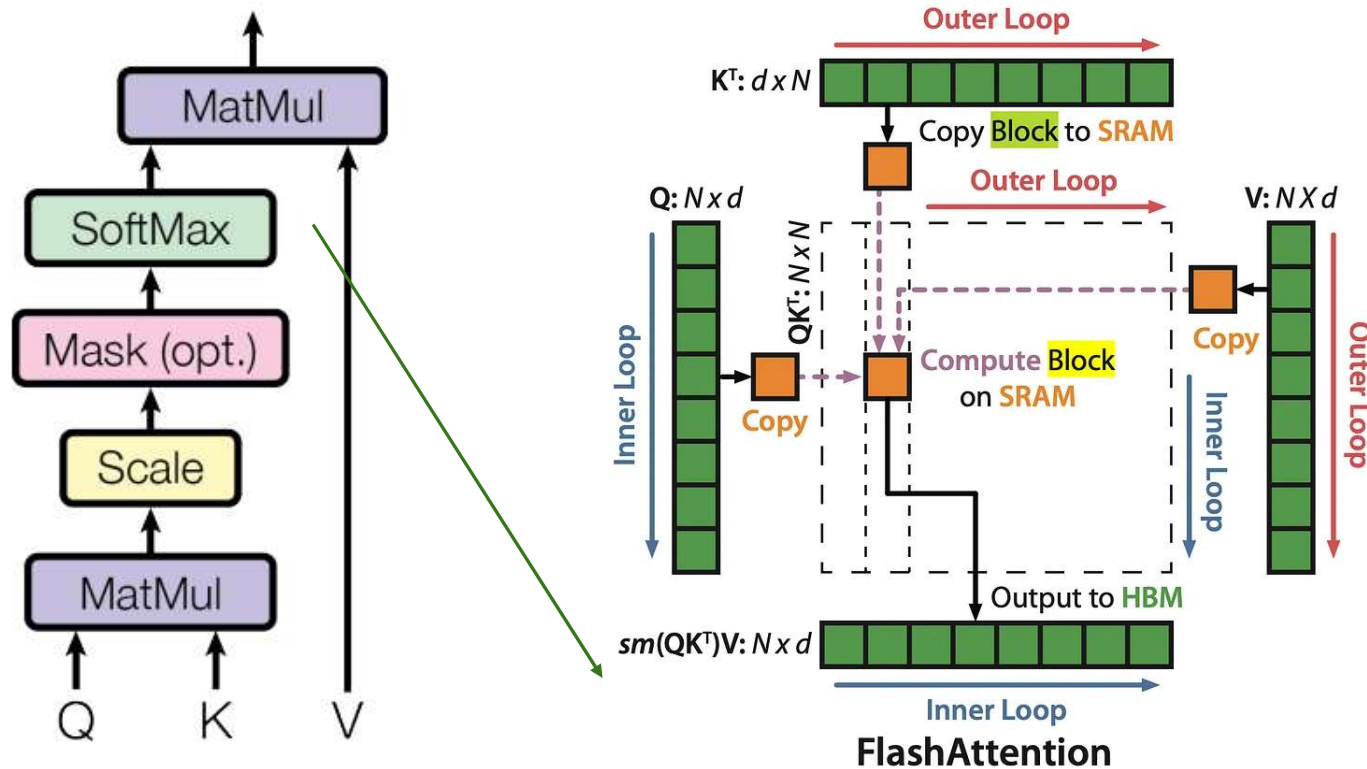
Gets computed
for every row -
Problem!



Year: 2022; NeurIPS

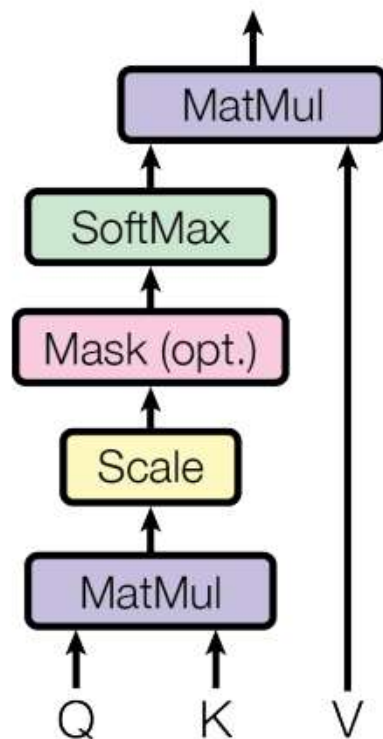


The magic does not end here! Tiling

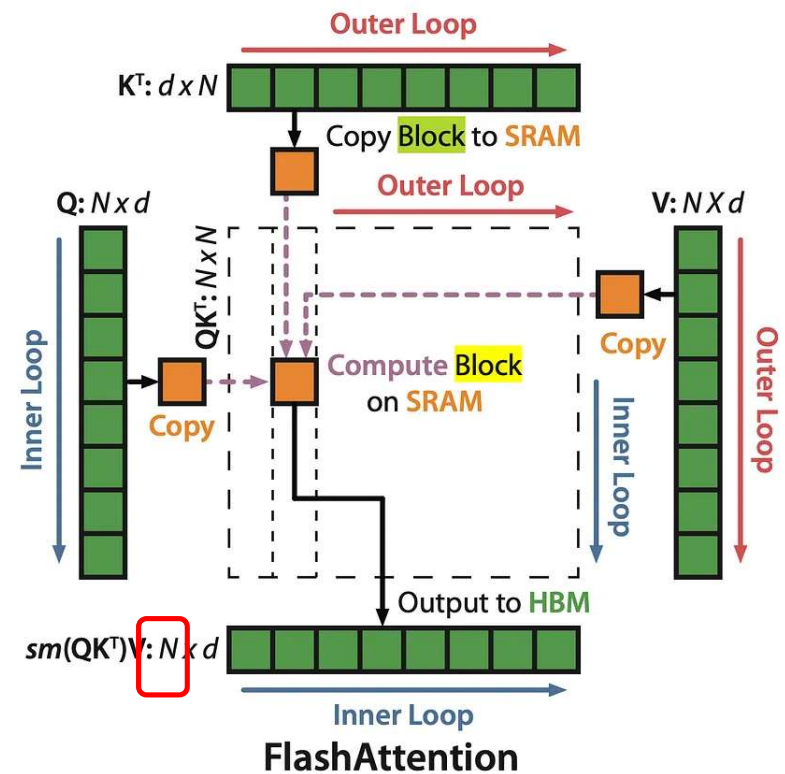




Does the story end here? What's the problem?

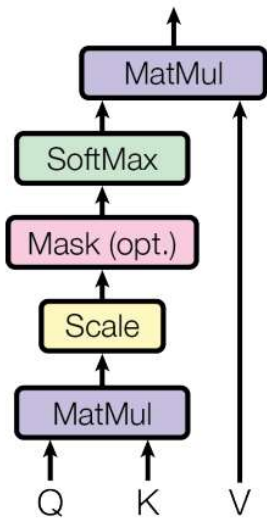


$$s(x_i) = \frac{e^{x_i}}{\sum_{j=1}^N e^{x_j}}$$





The softmax denominator problem



$$Q = [1] \quad K = [1, 2, 3] \quad V = [2, 4, 8]$$

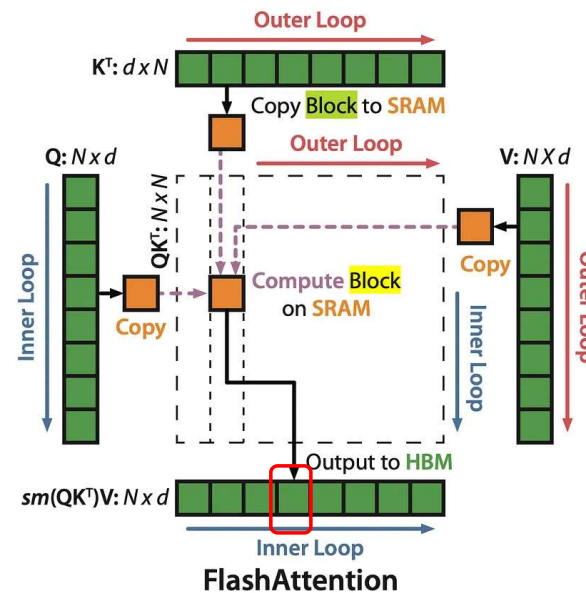
$$A = QK^T = [1, 2, 3]$$

$$V = [2, 4, 8]$$

$$O = \text{softmax}(A)V$$

$$O = \frac{N}{D} = \frac{2e^1 + 4e^2 + 8e^3}{e^1 + e^2 + e^3}$$

$$s(x_i) = \frac{e^{x_i}}{\sum_{j=1}^N e^{x_j}}$$



At $i = 0$

$$D_b = e^1$$

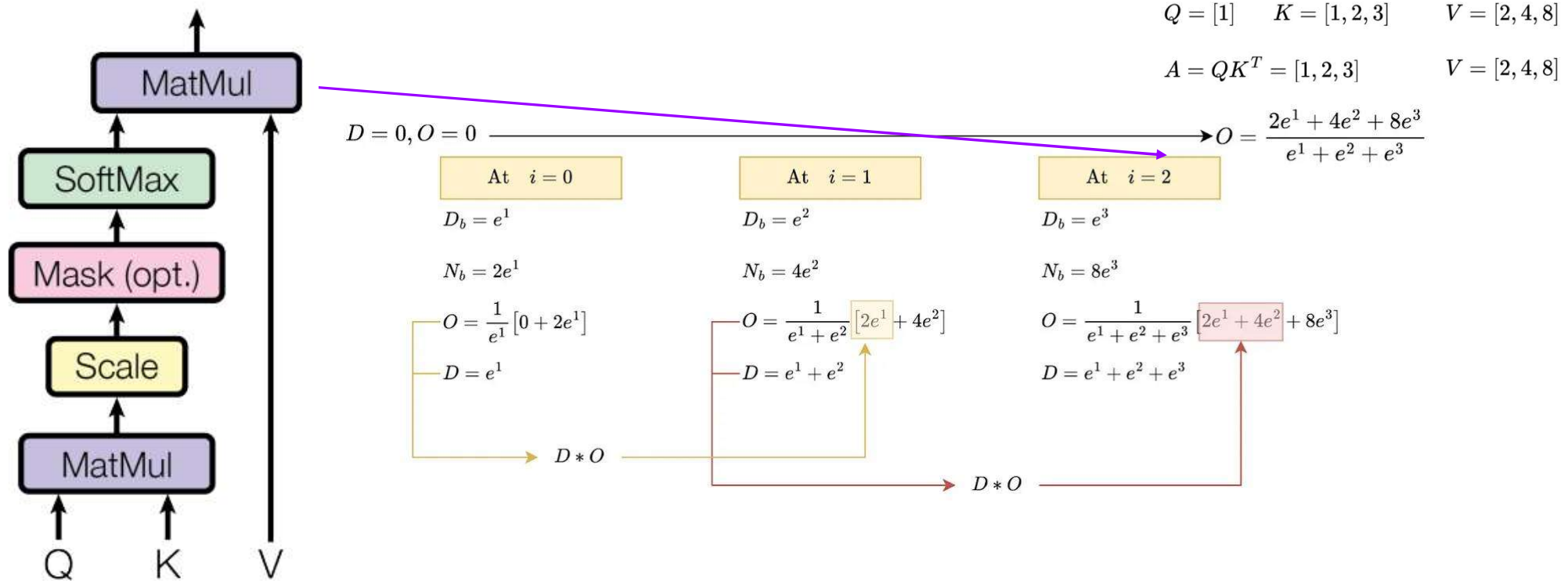
$$N_b = 2e^1$$

$$O = \frac{1}{e^1} [0 + 2e^1]$$



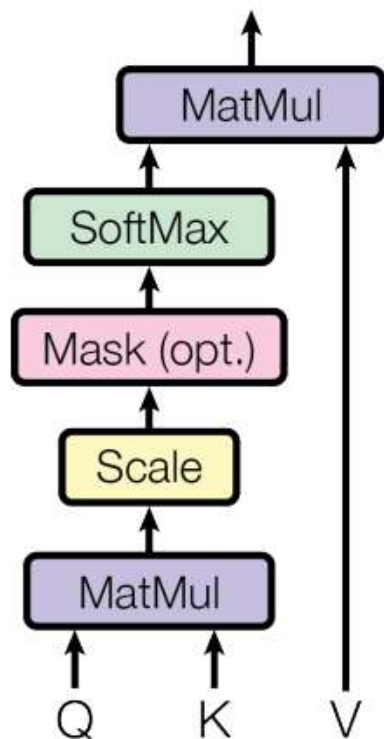


Summary Statistics - the final touch!





Summary Statistics - the final touch!



$$Q = [1] \quad K = [1, 2, 3] \quad V = [2, 4, 8]$$

$$A = QK^T = [1, 2, 3] \quad V = [2, 4, 8]$$

$$O = \text{softmax}(A)V$$

$$O = \frac{N}{D} = \frac{2e^1 + 4e^2 + 8e^3}{e^1 + e^2 + e^3}$$

$$D = 0, O = 0$$

Treat each element as a block,
so we have three blocks
for i in range(3):

$$D_b = \exp(Q[i] \times K[i])$$

$$N_b = V[i] * \exp(Q[i] \times K[i])$$

$$O = \frac{1}{D + D_b} [D * O + N_b]$$

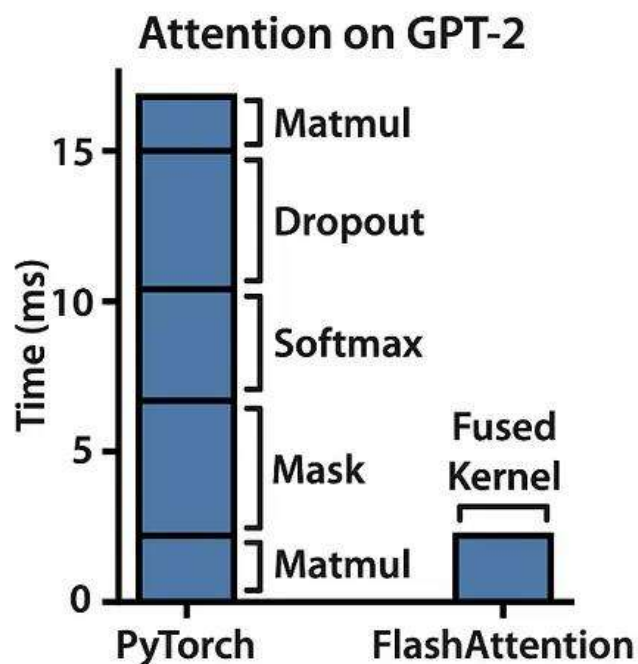
$$D = D + D_b$$



Year: 2022; NeurIPS



How well did they do?



BERT Implementation	Training time (minutes)
Nvidia MLPerf 1.1 [58]	20.0 ± 1.5
FLASHATTENTION (ours)	17.4 ± 1.4

*Time: $O(N*d)$*
*Space: $O(N*d)$*



So have we finally solved the attention hurdle?

- *Does your GPU come with?*
 - CUDA (or have to be re-written in ROCm (AMD) or SYCL (Intel))
 - Fast shared GPU memory (SRAM)
 - Tensor cores (specifically dedicated to matrix operations)
- Too much pro-NVIDIA (Ampere, Volta, etc.)
- A new attention on the block? Have to rewrite the fused kernel



Key Takeaways

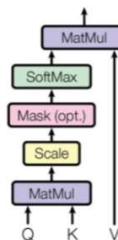
- Avoid unnecessary HBM read/write
- Maximize SRAM computation



Want more? Follow:

 Search 

[Browse State-of-the-Art](#) [Datasets](#) [Methods](#) [More](#) 



<https://paperswithcode.com/methods/category/attention-mechanisms>

